

CAPSTONE BOOTCAMP

(*SQL Database Development: Two-Week Learning & Implementation Report*)

Submitted By: Mohamed Yaseen J
Course: SQL / Database Management
Project: Capstone Project
Duration: Two Weeks
Submitted To: Hariharan J
Organization:
J.J. College of Engineering and
Technology

(Index)

S.NO	TOPIC
01	Introduction & Project Overview
02	Conceptual Analysis & ER Diagramming
03	ER Diagram Explanation and Diagram
04	SQL Introduction (Types + 1 Page Table)
05	Create Database
06	Create Table
07	Insert Data
08	Selecting Data (View)
09	DDL, DML, DCL, DQL
10	Aggregate Functions (MIN, MAX, AVG, SUM, COUNT, GROUP BY, HAVING, LIMIT)
11	Joins and Grouping
12	Window Functions
13	Proof of Documents (Installation)
14	Conclusion

Introduction

StyleRent is a Smart Clothing Rental & Inventory Management System.
The main purpose of this project is to make clothing rental simple, organized, and easy to manage using a database.

In this project, we learn how to design a database, create tables, store data, and retrieve information using SQL and PostgreSQL.

Project Overview

StyleRent manages different types of information required for a clothing rental system:

-  **Users** – stores customer details.
-  **Clothing** – stores clothing/product details.
-  **Category** – manages clothing categories.
-  **Rental** – stores rental details and dates.
-  **Rental Item** – connects rentals with clothing items.
-  **Payment** – stores payment information.
-  **Review** – stores customer ratings and comments.

Conceptual Analysis & ER Diagramming

ER Diagram

An ER Diagram (Entity Relationship Diagram) shows the entities, attributes, and relationships in a database.

It helps us understand which tables are connected and how the data is related.

Example:

Users → Rental → Clothing

Common SQL Keys

Primary Key (PK)

A Primary Key uniquely identifies each record in a table.

- No duplicate values
- Cannot be NULL

Example:

user_id

👉 Simple meaning: PK = Unique identity of each row.

Foreign Key (FK)

A Foreign Key connects one table to another table.

It usually refers to the Primary Key of another table.

Example:

rental.user_id → users.user_id

👉 Simple meaning: FK = Connects two tables.

Unique Key

A Unique Key does not allow duplicate values in a column.

Depending on the database system, it can generally allow NULL values, unlike a Primary Key.

Example:

email

👉 **Simple meaning: Unique = No duplicate values.**

Composite Key

A Composite Key uses two or more columns together to uniquely identify a record.

Each column does not have to be unique by itself.

Example:

rental_id + clothing_id

👉 **Simple meaning: Composite Key = Multiple columns together make one unique key.**

Before writing any code, we visualize the business requirements using Chen's Database Notation.

ER Component	Explanation
Entity (Rectangle)	The real-world objects, such as Users, Rentals and Clothing.
Attribute (Oval)	The data we store about an entity, such as Name, Email and Price.
Relationship (Diamond)	How entities connect to each other, such as Makes, Has and Contains.
Cardinality	The mathematical nature of a connection: 1:1, 1:N or M:N.

Example: 1 User can make Many (N) Rentals. Therefore, the relationship between USER and RENTAL is represented as 1:N.

STYLERENT _ ER DIAGRAM

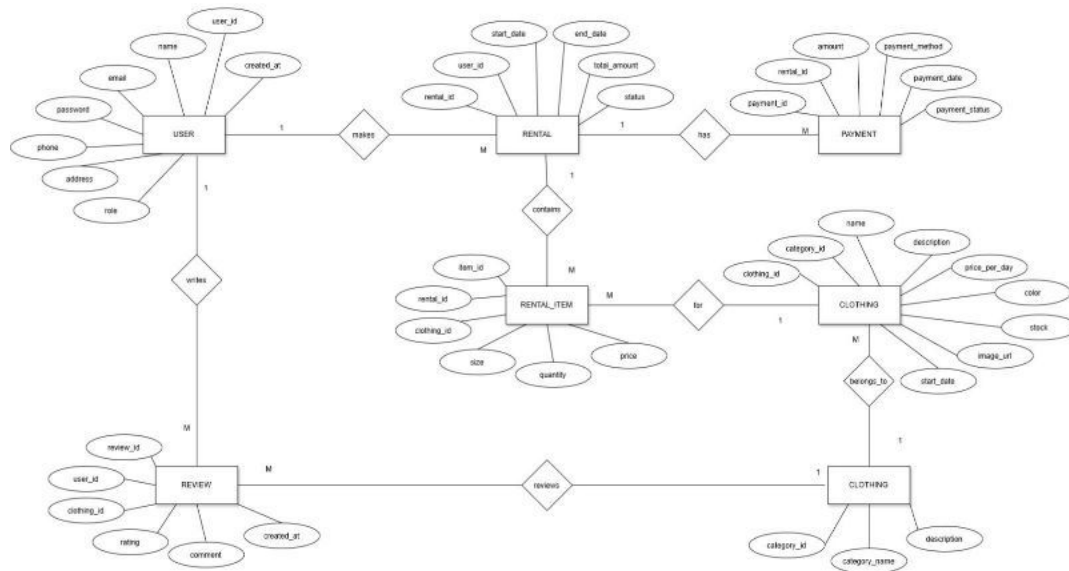


Figure 1. StyleRent ER Diagram designed using Draw.i

Entity Overview

Entity	Purpose
USER	Stores user account and profile information.
RENTAL	Stores rental transaction and date details.
PAYMENT	Stores payment details associated with rentals.
RENTAL_ITEM	Connects rentals and clothing items and stores item-level rental details.
CLOTHING	Stores clothing/product information.
CATEGORY	Stores clothing category information.
REVIEW	Stores user ratings, comments and review information.

Relationship and Cardinality

USER → makes → RENTAL : 1:N

RENTAL → has → PAYMENT : 1:N

RENTAL → contains → RENTAL_ITEM : 1:N

RENTAL_ITEM → for → CLOTHING : M:1

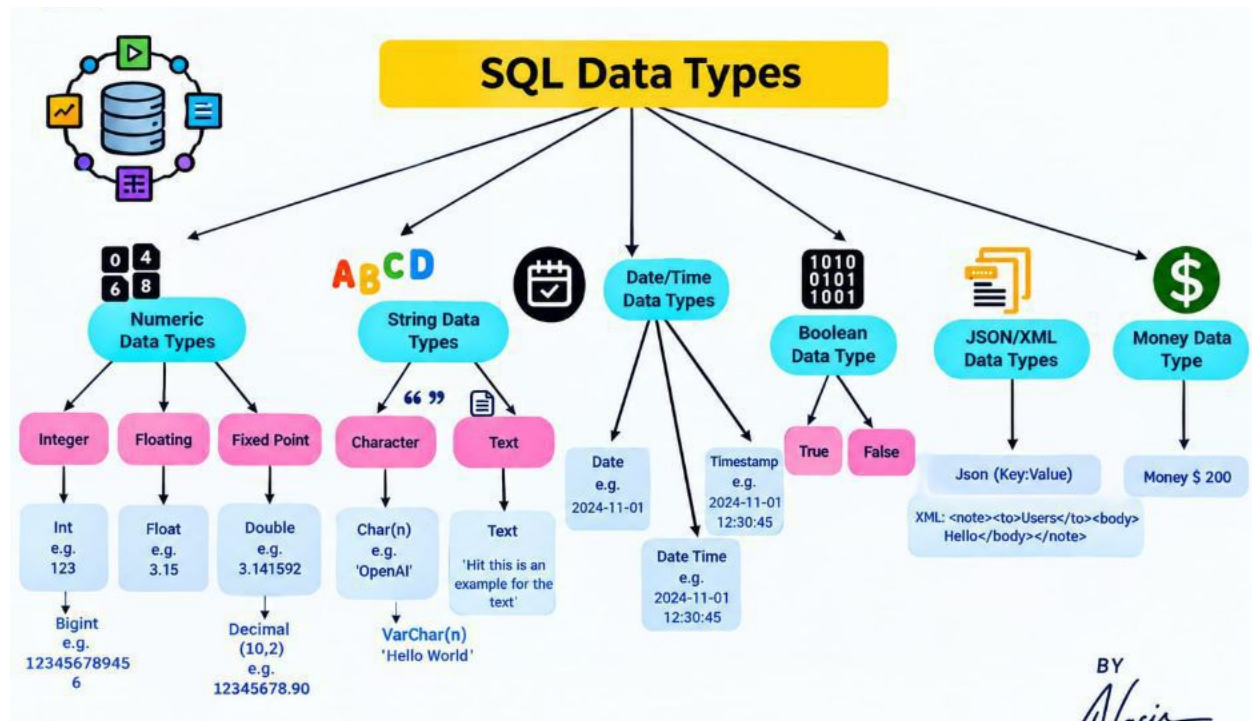
CLOTHING → belongs_to → CATEGORY : M:1

USER → writes → REVIEW : 1:N

REVIEW → reviews → CLOTHING : M:1

SQL – Structured Query Language

SQL is a language used to communicate with a database.



We use SQL to:

Create data

Insert data

Read data

Update data

Delete data

Example:

```
SELECT * FROM users;
```

👉 SQL is a language, just like Python is a programming language.

MySQL

MySQL is a database management software that uses SQL.

It helps us to:

Store data

Manage data

Retrieve data

Modify data

Simple example:

SQL → Language

MySQL → Software that uses SQL

DBMS – Database Management System

DBMS is a system used to store, manage, and access data in a database.

Main types:

DBMS ↓

1.RDBMS

2.NoSQL / Non-Relational Database

1. RDBMS – Relational Database Management System

RDBMS stores data in the form of tables.

A table contains:

Rows → Records

Columns → Fields/Attributes

Example:

user_id

name

email

1

Arun

arun@gmail.com

2

Priya

priya@gmail.com

Common RDBMS:

MySQL

PostgreSQL

Oracle Database

Microsoft SQL Server

👉 StyleRent can use an RDBMS because its data is organized into tables such as users, clothing, and rental.

2. NoSQL / Non-Relational Database

NoSQL databases do not require data to be stored only in traditional rows and columns. They are useful for flexible or large-scale data.

Examples:

MongoDB

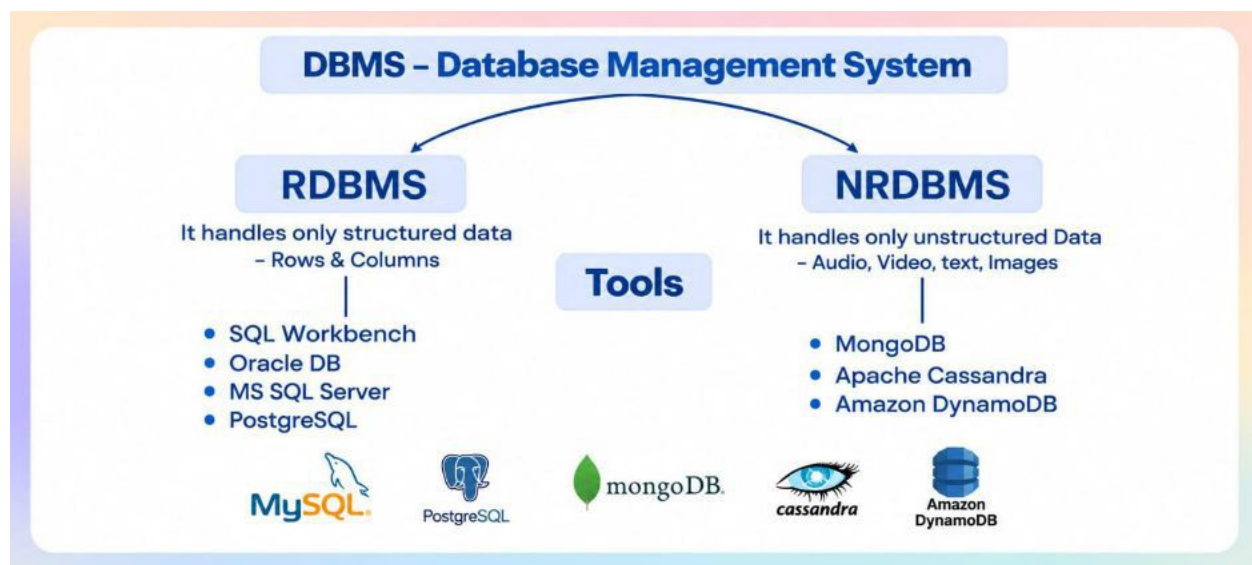
Apache Cassandra

Amazon DynamoDB

They can handle different types of data such as:

Text, Images, Audio

Video, JSON documents



MySQL - Create DataBase

```
1 CREATE DATABASE stylerentdb;
2 USE stylerentdb;
3
```

Create Table

```
1 • CREATE DATABASE stylerentdb;
2   USE stylerentdb;
3
4   CREATE TABLE user(
5       user_id INT PRIMARY KEY,
6       user_name VARCHAR(100) NOT NULL,
7       phone VARCHAR(20) NOT NULL,
8       email VARCHAR(50) UNIQUE NOT NULL,
9       address VARCHAR(255)
10  );
11
12 • CREATE TABLE clothing(
13     clothing_id INT PRIMARY KEY,
14     category_id INT NOT NULL,
15     name VARCHAR(100) NOT NULL,
16     description VARCHAR(300) NOT NULL
17 );
18
15     name VARCHAR(100) NOT NULL,
16     description VARCHAR(300) NOT NULL
17 );
18
19 • CREATE TABLE rental(
20     rental_id INT PRIMARY KEY,
21     user_id INT NOT NULL,
22     clothing_id INT NOT NULL,
23     start_date DATE NOT NULL,
24     end_date DATE NOT NULL,
25     total_amount DECIMAL(10,2) NOT NULL,
26     status VARCHAR(30) NOT NULL,
27     FOREIGN KEY (user_id) REFERENCES user (user_id),
28     FOREIGN KEY (clothing_id) REFERENCES clothing(clothing_id)
29 );
30
31
```

Insert Datas

```
INSERT INTO user (user_id, user_name, phone, email, address) VALUES
(1, 'Arun Kumar', '9876543210', 'arun@gmail.com', 'Chennai'),
(2, 'Priya Devi', '9876543211', 'priya@gmail.com', 'Coimbatore'),
(3, 'Rahul Raj', '9876543212', 'rahul@gmail.com', 'Madurai'),
(4, 'Fathima Noor', '9876543213', 'fathima@gmail.com', 'Trichy'),
(5, 'Karthik S', '9876543214', 'karthik@gmail.com', 'Salem');
```

Selecting Data

59 • `SELECT * FROM rental;`

rental_id	user_id	clothing_id	start_date	end_date	total_amount	status
1	1	1	2026-09-01	2026-09-03	1500.00	Completed
2	2	2	2026-09-05	2026-09-07	1000.00	Active
3	3	3	2026-09-10	2026-09-12	2000.00	Booked
4	4	4	2026-09-15	2026-09-17	1200.00	Booked
5	5	5	2026-09-20	2026-09-22	1800.00	Pending



In SQL, there are four main categories of SQL statements, each serving a different purpose:



Data Definition Language (DDL)

DDL is used to define the **structure** of a database, such as creating, modifying, and deleting database objects like tables, indexes, and constraints.

Main commands:

- **CREATE** (create)
- **ALTER** (modify)
- **DROP** (delete)

Example:

```
CREATE TABLE Students (  
  StudentID INT,  
  Name VARCHAR(50),  
  Age INT  
);
```



Data Manipulation Language (DML)

DML is used to add, change, or delete **data** inside a table.

Main commands:

- **INSERT** (add)
- **UPDATE** (change)
- **DELETE** (remove)

Example:

```
INSERT INTO Students  
(StudentID, Name, Age)  
VALUES (1, 'John', 20);
```



Data Control Language (DCL)

DCL is used to control who can **access** or use the database.

Main commands:

- **GRANT** (give permission)
- **REVOKE** (take permission)

Example:

```
GRANT SELECT ON Students  
TO user1;  
  
REVOKE SELECT ON Students  
FROM user1;
```



Data Query Language (DQL)

DQL is used to **retrieve** or **read** data from the database.

Main command:

- **SELECT** (retrieve)

Example:

```
SELECT * FROM Students;  
  
SELECT Name, Age  
FROM Students  
WHERE Age > 18;
```

1. DDL – Data Definition Language 🏗️

Easy meaning: DDL is used to create or change the structure of a database.

Think: “Build the table.”

Main commands:

CREATE → create something new

ALTER → change the structure

DROP → remove a database object

Example:

```
CREATE TABLE Students (  
    StudentID INT,  
    Name VARCHAR(50),  
    Age INT  
);
```

👉 This creates a Students table.

If you want to add an Email column:

```
ALTER TABLE Students  
ADD Email VARCHAR(100);  
Remember: DDL = Structure
```

2. DML – Data Manipulation Language

Easy meaning: DML is used to add, change, or delete data inside a table.

Think: “Work with the data.”

Main commands:

INSERT → add data

UPDATE → change data

DELETE → remove data

Example:

Add a student:

INSERT INTO Students (StudentID, Name, Age)

VALUES (1, 'John', 20);

Change John's age:

UPDATE Students

SET Age = 21

WHERE StudentID = 1;

Delete John:

DELETE FROM Students

WHERE StudentID = 1;

Remember: DML = Data

3. DCL – Data Control Language

Easy meaning: DCL is used to control who can access or use the database.

Think: “Who is allowed?”

Main commands:

GRANT → give permission
REVOKE → take permission away

Example:

Give a user permission to read the Students table:

GRANT SELECT ON Students TO user1;

Take that permission back:

REVOKE SELECT ON Students FROM user1;

Remember: DCL = Permission

4. DQL – Data Query Language

Easy meaning: DQL is used to retrieve/read data from a database.

The main command is:

SELECT

Example:

Get all students:

SELECT * FROM Students;

Get only student names:

SELECT Name FROM Students;

Get students older than 18:

SELECT *

FROM Students

WHERE Age > 18;

Remember: DQL = Query/Read

Type	Think	Main Command
DDL	 Structure	CREATE
DML	 Data	INSERT
DCL	 Permission	GRANT
DQL	 Read	SELECT

One simple story:

Imagine a **Students** database:

1. 🛠️ **DDL** → Build the Students table.
2. ✍️ **DML** → Add/change/delete student information.
3. 🔍 **DQL** → Ask the database, "Show me the students."
4. 🗝️ **DCL** → Decide who is allowed to see or modify the students.

Aggregate Functions

Aggregate functions perform calculations on multiple rows and return one result.

Examples: **SUM()**, **AVG()**, **COUNT()**, **MAX()**, **MIN()**.



Sequence of Execution

SQL does not execute clauses in the order you write them.
It follows a fixed sequence to get the correct result.

Step	Clause	Purpose
1	FROM	Specifies the data source (tables, views, joins). Resolves the tables and retrieves initial data.
2	WHERE	Filters rows based on the specified condition.
3	GROUP BY	Groups rows into subsets based on one or more columns for aggregation.
4	HAVING	Filters groups created by GROUP BY based on a condition.
5	SELECT	Selects specific columns, calculated expressions, or aggregated values for the output.
6	ORDER BY	Sorts the result set based on specified columns in ascending (ASC) or descending (DESC) order.
7	LIMIT	Restricts the number of rows returned in the result set.

**Example**

For the query below, SQL will execute the clauses in the sequence shown (1 → 7).

```
SELECT department, COUNT(*) AS total
FROM employees
WHERE salary > 50000
GROUP BY department
HAVING COUNT(*) > 2
ORDER BY total DESC
LIMIT 5;
```

Execution Flow (1 → 7)
1 → 2 → 3 → 4 → 5 → 6 → 7
FROM (tables) **WHERE** (filter rows) **GROUP BY** (group data) **HAVING** (filter groups) **SELECT** (choose cols) **ORDER BY** (sort) **LIMIT** (top rows)

✓ Remember: FROM → WHERE → GROUP BY → HAVING → SELECT → ORDER BY → LIMIT

```

59 • SELECT * FROM rental;
60 • SELECT clothing_id, category_id, name, description FROM clothing;

```

Result Grid	Filter Rows:	Edit:	Export/Import:	Wrap Cell Cont
clothing_id	category_id	name	description	
1	1	Red Evening Gown	Elegant red gown for parties and special occasions	
2	2	Blue Denim Jacket	Stylish blue denim jacket for casual wear	
3	3	Black Formal Suit	Classic black suit for professional occasions	
4	4	Floral Maxi Dress	Beautiful floral maxi dress for casual outings	
5	5	Traditional Saree	Elegant traditional saree for special events	

```

64 • SELECT clothing_id ,category_id, name FROM clothing
65     order by clothing_id desc
66     limit 2;
67

```

Result Grid	Filter Rows:	Edit:	Export/Import:	Wrap Cell
clothing_id	category_id	name		
5	5	Traditional Saree		
4	4	Floral Maxi Dress		
NULL	NULL	NULL		

```

64 • SELECT clothing_id ,category_id, name FROM clothing
65     order by clothing_id desc;
66

```

Result Grid	Filter Rows:	Edit:	Export/Import:	Wrap Cell
clothing_id	category_id	name		
5	5	Traditional Saree		
4	4	Floral Maxi Dress		
3	3	Black Formal Suit		
2	2	Blue Denim Jacket		
1	1	Red Evening Gown		

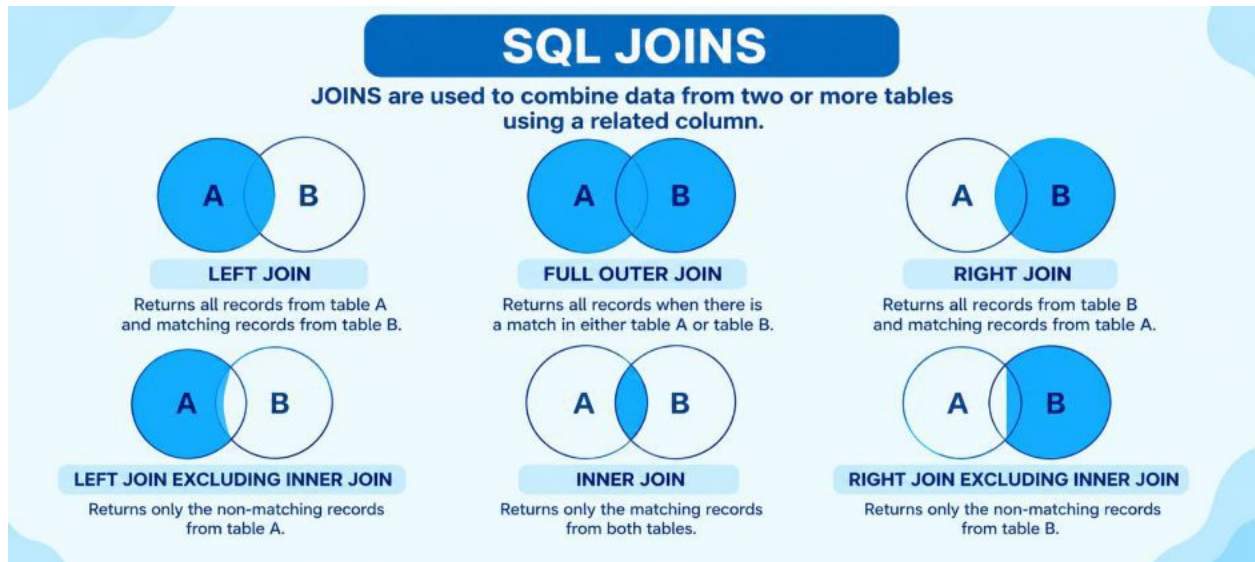
```

62 • SELECT clothing_id, category_id, name, description FROM clothing
63     WHERE category_id<4;
64

```

Result Grid	Filter Rows:	Edit:	Export/Import:	Wrap Ce
clothing_id	category_id	name	description	
1	1	Red Evening Gown	Elegant red gown for parties and special occasions	
2	2	Blue Denim Jacket	Stylish blue denim jacket for casual wear	
3	3	Black Formal Suit	Classic black suit for professional occasions	
NULL	NULL	NULL	NULL	

Joins AND Grouping



JOIN is used to combine data from two or more tables using a related column.

For example:

- **users** → user details
- **rental** → rental details
- Common column → **user_id**

1. INNER JOIN

Meaning: Shows only the records that match in both tables.

```
SELECT u.name, r.rental_id, r.total_amount
FROM users u
INNER JOIN rental r
ON u.user_id = r.user_id;
```

Simple:

👉 User-ku rental irundha mattum result-la varum.

2. LEFT JOIN

Meaning: Shows all records from the left table and matching records from the right table.

```
SELECT u.name, r.rental_id
FROM users u
```

```
LEFT JOIN rental r
ON u.user_id = r.user_id;
```

Simple:

👉 All users varuvanga. Rental illana **NULL** varum.

3. RIGHT JOIN

Meaning: Shows all records from the right table and matching records from the left table.

```
SELECT u.name, r.rental_id
FROM users u
RIGHT JOIN rental r
ON u.user_id = r.user_id;
```

Simple:

👉 All rental records varum. User match illana **NULL** varum.

4. FULL OUTER JOIN

Meaning: Shows all records from both tables, whether they match or not.

```
SELECT u.name, r.rental_id
FROM users u
FULL OUTER JOIN rental r
ON u.user_id = r.user_id;
```

Important:

👉 MySQL-la **FULL OUTER JOIN** directly support aagathu.
PostgreSQL-la direct-a use pannalam.

For MySQL, use:

```
SELECT u.name, r.rental_id
FROM users u
LEFT JOIN rental r
ON u.user_id = r.user_id
```

UNION

```
SELECT u.name, r.rental_id
```

```
FROM users u
RIGHT JOIN rental r
ON u.user_id = r.user_id;
```

5. CROSS JOIN

Meaning: Every row in first table combines with every row in second table.

```
SELECT u.name, r.rental_id
FROM users u
CROSS JOIN rental r;
```

Simple:

👉 5 users × 5 rentals = 25 combinations.

6. SELF JOIN

Meaning: A table joins with itself.

Example:

```
SELECT a.name AS User1, b.name AS User2
FROM users a
JOIN users b
ON a.user_id <> b.user_id;
```

Simple:

👉 Same **users** table-la irukkura records-a compare panna use pannuvom.

Window Function

The Rule

- **GROUP BY** is like taking a team photo where only the team captain shows up to represent the whole team. You lose individual details.
- Window Function keeps every single person in the photo and just adds a calculation next to their name.

Visual Example

Suppose you have 3 employees in Sales:

Name	Department	Salary
Alice	Sales	\$5,000
Bob	Sales	\$6,000
Charlie	Sales	\$4,000

1. Using **GROUP BY**

If you calculate total salary with **GROUP BY Department**, you lose the names:

Department	Total Salary
Sales	\$15,000

2. Using a Window Function

If you use **SUM(Salary) OVER (PARTITION BY Department)**, everyone stays on screen:

Name	Department	Salary	Department Total
Alice	Sales	\$5,000	\$15,000
Bob	Sales	\$6,000	\$15,000

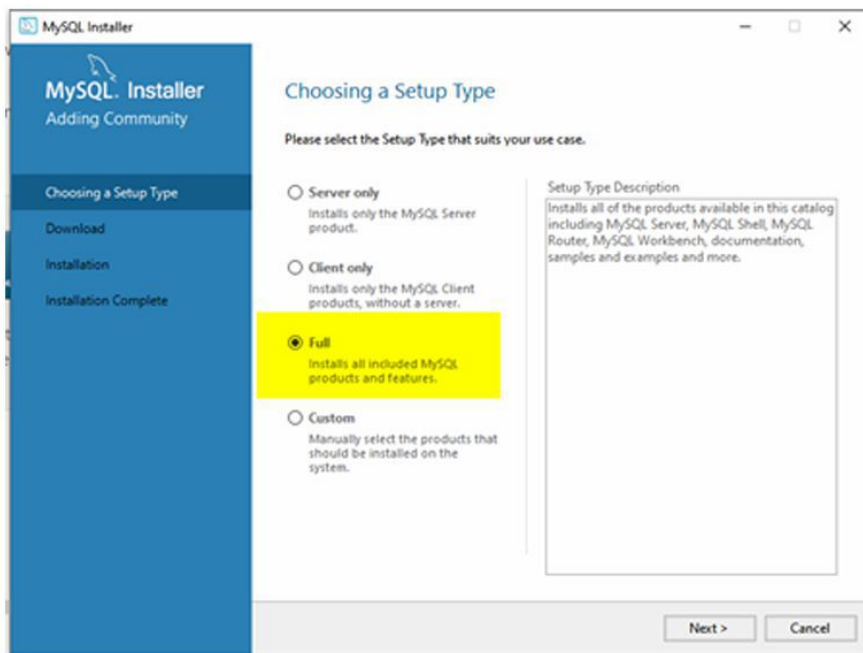
Charlie	Sales	\$4,000	\$15,000
---------	-------	---------	----------

The Magic Keyword

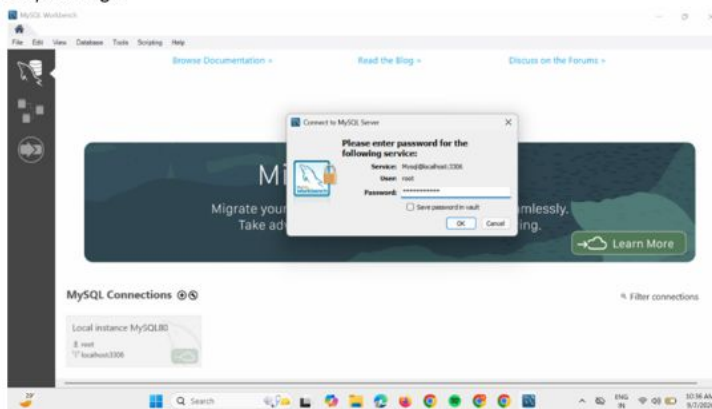
Whenever you see **OVER ()**, you are using a Window Function. It tells SQL: *"Calculate this across a group of rows, but don't hide my individual rows!"*

Installation Proof

Step 1: Open Mysql



Step 2: Login



Conclusion

The Capstone Project helped me use what I learned in a practical way. I learned how to work with SQL, databases, tables, queries, joins, and data analysis.

Overall, this project improved my technical and problem-solving skills and gave me confidence to work on real-world projects.